



EGE ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ
2025-2026

STM32 ve Bluetooth Kontrollü Robotik
Araç Tasarımı

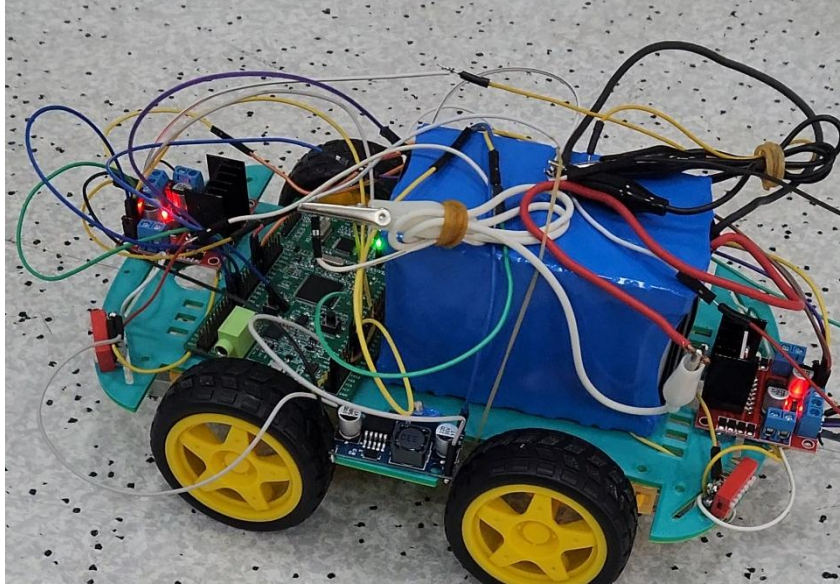
Ad Soyad : Sam ZOGHI
: Amirkia MAHD GHAREHBAGH
: Macit CEM KIZIL
Öğrenci No : 05210000126
: 05210000108
: 05210000665
Grup Numarası : 25
Tarih : 25.12.2025

İçindekiler

Proje Özeti	3
Giriş ve Amaç	3
Donanım Mimarisi ve Malzeme Listesi	3
Çevre Birimlerinin Konfigürasyonu	4
Kontrol Algoritması	4
Çift Yönlü Haberleşme (Feedback)	5
Sonuç ve Değerlendirme	5
Yazılım Kod Analizi	5

Proje Özeti

Bu projede, gömülü sistemler mimarisi kullanılarak, Android tabanlı bir mobil cihaz üzerinden Bluetooth haberleşmesi ile uzaktan kontrol edilebilen, dört tekerlekten çekişli (4WD) bir robotik araç tasarlanmış ve gerçekleştirilmiştir. Sistemin ana kontrolcüsü olarak STM32 Discovery geliştirme kartı kullanılmıştır. Araç, ileri, geri, sağa, sola manevra kabiliyetlerinin yanı sıra, PWM sinyalleri ile hız kontrolü yapabilme yeteneğine sahiptir.



Giriş ve Amaç

Projenin temel amacı, mikrodenetleyici tabanlı sistemlerin gerçek zamanlı kontrol uygulamalarında nasıl kullanıldığını göstermektir. Bu kapsamda UART haberleşme protokolü, Timer kesmeleri, PWM sinyal üretimi ve H-Köprüsü motor sürücü mimarileri pratik olarak uygulanmıştır.

Donanım Mimarisi ve Malzeme Listesi

Projede kullanılan donanım bileşenleri, güç gereksinimleri ve sinyal işleme kapasitelerine göre seçilmiştir:

- **Mikrodenetleyici:** STM32 Discovery Kartı (STM32F4 serisi). Yüksek işlem gücü ve geniş I/O portları nedeniyle tercih edilmiştir.
- **Motor Sürücü:** 2 Adet L298N. Her bir sürücü, motorların yön ve hız kontrolünü sağlamak için kullanılmıştır. L298N entegresi, mikrodenetleyiciden gelen düşük akımlı mantıksal sinyalleri (GPIO), motorları sürmek için gereken yüksek akıma çevirir.
- **Aktüatörler:** 4 Adet TT DC Redüktörlü Motor ve 4 Adet Tekerlek.
- **Haberleşme:** HC-05 Bluetooth Modülü. UART protokolü üzerinden seri haberleşme sağlar. Bluetooth haberleşmesi USART2 üzerinden, **9600 baudrate** ve kesme tabanlı (interrupt-driven) olarak gerçekleştirilmiştir.

- **Güç Kaynağı:** 12V 3S 5P 18650 Li-ion Batarya Paketi (10A BMS devreli). Yüksek akım kapasitesi ile uzun süreli çalışma sağlar.
- **Voltaj Regülasyonu:** LM2596 Voltaj Regülatörü. Bataryadan gelen 12V gerilimi, STM32 ve Bluetooth modülünün güvenli çalışması için 5V seviyesine düşürür.
- **Şasi:** Mekanik montajın yapıldığı ana gövde.

Çevre Birimlerinin Konfigürasyonu

- **UART (Evrensel Asenkron Alıcı-Verici):** USART2 birimi 9600 baud rate hızında yapılandırılmıştır. İşlemciyi meşgul etmemek (non-blocking) adına veri alımı "Interrupt" (Kesme) modunda (HAL_UART_Receive_IT) gerçekleştirilmiştir.
- **PWM (Hız Kontrolü):** TIM1 zamanlayıcısı PWM modunda ayarlanmıştır. current_speed değişkeni ile TIM_CHANNEL_1 üzerindeki doluluk oranı (duty cycle) değiştirilerek motorlara giden ortalama voltaj ve dolayısıyla hız kontrol edilmiştir.
 - $TIM1\ clock = APB2 \times 2 = 168\ MHz$
 - $Timer\ clock = 168\ MHz / 84 = 2\ MHz$
 - $PWM\ frekansı = 2\ MHz / 1000 = 2\ kHz$
 - PSC = 84 - 1 ve ARR = 1000 - 1 olarak ayarlanmıştır.
- **GPIO (Giriş/Çıkış):** GPIOD ve GPIOE portları motor sürücülerin yön pinlerini (IN1, IN2, IN3, IN4) kontrol etmek için çıkış olarak ayarlanmıştır.

Kontrol Algoritması

Bluetooth üzerinden gelen karakter verisine (bt_rx) göre switch-case yapısı kullanılarak aracın hareketi belirlenir:

- **'F' (Forward):** Tüm motorlar ileri yönde aktif edilir.
- **'B' (Backward):** Tüm motorlar geri yönde aktif edilir.
- **'L' (Left):** Sağ motorlar ileri dönerken, sol motorlar durdurulur. Bu sayede araç olduğu yerde sola dönme hareketi yapar.
- **'R' (Right):** Sol motorlar ileri dönerken, sağ motorlar durdurulur.
- **'S' (Stop):** PWM sinyali kesilir ve tüm GPIO pinleri RESET konumuna çekilerek araç durdurulur.
- **'U' (Up) ve 'D' (Down):** Hız artırma ve azaltma komutlarıdır. PWM değeri 50'şer birimlik adımlarla değiştirilir.

Çift Yönlü Haberleşme (Feedback)

Sistem sadece veri almaz, aynı zamanda kullanıcıya geri bildirimde bulunur. 'U' veya 'D' komutları geldiğinde, güncel hız yüzdesi hesaplanır ve sprintf ile formatlanarak HAL_UART_Transmit fonksiyonu ile mobil uygulamaya metin olarak gönderilir.

Sonuç ve Değerlendirme

Bu proje sonucunda, gömülü yazılım ve donanım bileşenleri başarılı bir şekilde entegre edilmiştir. 12V'luk güçlü batarya sistemi ve L298N sürücüler sayesinde araç zorlu zeminlerde dahi hareket kabiliyetini korumuştur. Yazılım tarafında kullanılan "Interrupt" yapısı sayesinde veri kaybı olmadan akıcı bir kontrol deneyimi sağlanmıştır. Alınan Bluetooth komutu tek karakterliktir ve her işlenişten sonra temizlenmektedir, bu sayede bağlantı kopması veya telefonun kapanması gibi durumlarda, araç güvenlik nedeni ile durur ve hareket etmez.

Yazılım Kod Analizi

Projenin beyni olan main.c dosyasındaki kritik fonksiyonlar şunlardır:

- **HAL_UART_RxCpltCallback:** Veri geldiği anda tetiklenen kesme fonksiyonudur. Gelen veriyi işleme alır ve UART hattını tekrar dinlemeye açar.
- **Motor_Forward/Backward/Left/Right:** GPIO pinlerinin Lojik-1 (SET) ve Lojik-0 (RESET) durumlarını ayarlayarak L298N üzerindeki H-Köprüsünü yönlendirir.
- **Motor_Speed_Up/Down:** PWM darbe genişliğini (Pulse) güvenli sınırlar (PWM_MIN - PWM_MAX) içerisinde artırıp azaltır.

```
#include "main.h"
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
TIM_HandleTypeDef htim1;
```

```
UART_HandleTypeDef huart2;
```

```
volatile uint8_t ileri_basili = 0;
```

```
volatile uint8_t geri_basili = 0;
```

```
volatile uint16_t current_speed = 500;
```

```
char msg_buffer[50];                // Global buffer
volatile uint8_t bt_rx;             // Bluetooth'tan gelen ham veri
```

```
#define PWM_MIN 200
```

```
#define PWM_MAX 950
```

```
#define PWM_STEP 50
```

```
void Motor_Stop(void);
```

```
void Motor_Forward(void);
```

```
void Motor_Backward(void);
```

```
void Motor_Left(void);
```

```
void Motor_Right(void);
```

```
void Motor_Speed_Up(void);
```

```
void Motor_Speed_Down(void);
```

```
int main(void)
```

```
{
```

```
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
```

```
    HAL_UART_Receive_IT(&huart2, (uint8_t *)&bt_rx, 1);
```

```
    while (1)
```

```
    {
```

```
        switch (bt_rx)
```

```
        {
```

```
            case 'F': Motor_Forward(); break; // Forward
```

```
            case 'B': Motor_Backward(); break; // Backward
```

```
            case 'L': Motor_Left();    break; // Left
```

```
            case 'R': Motor_Right();   break; // Right
```

```
            case 'S': Motor_Stop();    break; // Stop
```

```
            case 'U': Motor_Speed_Up(); break; // Stop
```

```

        case 'D': Motor_Speed_Down();    break; // Stop

        default: Motor_Stop();    break;
    }

    HAL_Delay(10);        // Döngüyü çok hızlı döndürüp işlemciyi yormayalım
}

}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        // Veri geldiği anda buraya düşer.

        HAL_UART_Receive_IT(&huart2, (uint8_t *)&bt_rx, 1);        // Tekrar dinlemeye geç
    }
}

void Motor_Stop(void)
{
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);

    // Tüm yön pinlerini sıfırla

    HAL_GPIO_WritePin(GPIOD, on_sag_in1_Pin | on_sol_in1_Pin | arka_sag_in1_Pin |
        arka_sol_in1_Pin, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(GPIOE, on_sag_in2_Pin | on_sol_in2_Pin | arka_sag_in2_Pin |
        arka_sol_in2_Pin, GPIO_PIN_RESET);
}

void Motor_Forward(void)
{
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, current_speed);

    // Sol ve Sağ motorlar İLERİ

    HAL_GPIO_WritePin(GPIOD, on_sag_in1_Pin | on_sol_in1_Pin | arka_sag_in1_Pin |
        arka_sol_in1_Pin, GPIO_PIN_SET);
}

```

```
    HAL_GPIO_WritePin(GPIOE, on_sag_in2_Pin | on_sol_in2_Pin | arka_sag_in2_Pin |  
    arka_sol_in2_Pin, GPIO_PIN_RESET);  
}
```

void Motor_Backward(void)

```
{  
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, current_speed);  
    // Sol ve Sağ motorlar GERİ  
    HAL_GPIO_WritePin(GPIOD, on_sag_in1_Pin | on_sol_in1_Pin | arka_sag_in1_Pin |  
    arka_sol_in1_Pin, GPIO_PIN_RESET);  
    HAL_GPIO_WritePin(GPIOE, on_sag_in2_Pin | on_sol_in2_Pin | arka_sag_in2_Pin |  
    arka_sol_in2_Pin, GPIO_PIN_SET);  
}
```

void Motor_Left(void)

```
{  
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, current_speed);  
    // Sağ motorlar İLERİ  
    HAL_GPIO_WritePin(GPIOD, on_sag_in1_Pin | arka_sag_in1_Pin, GPIO_PIN_SET);  
    HAL_GPIO_WritePin(GPIOE, on_sag_in2_Pin | arka_sag_in2_Pin, GPIO_PIN_RESET);  
    // Sol motorlar DUR  
    HAL_GPIO_WritePin(GPIOD, on_sol_in1_Pin | arka_sol_in1_Pin, GPIO_PIN_RESET);  
    HAL_GPIO_WritePin(GPIOE, on_sol_in2_Pin | arka_sol_in2_Pin, GPIO_PIN_RESET);  
}
```

void Motor_Right(void)

```
{  
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, current_speed);  
    // Sol motorlar İLERİ  
    HAL_GPIO_WritePin(GPIOD, on_sol_in1_Pin | arka_sol_in1_Pin, GPIO_PIN_SET);  
    HAL_GPIO_WritePin(GPIOE, on_sol_in2_Pin | arka_sol_in2_Pin, GPIO_PIN_RESET);  
    // Sağ motorlar DUR  
    HAL_GPIO_WritePin(GPIOD, on_sag_in1_Pin | arka_sag_in1_Pin, GPIO_PIN_RESET);
```

```
    HAL_GPIO_WritePin(GPIOE, on_sag_in2_Pin | arka_sag_in2_Pin, GPIO_PIN_RESET);  
}
```

```
void Motor_Speed_Up(void)
```

```
{  
    if (current_speed + PWM_STEP <= PWM_MAX)  
        current_speed += PWM_STEP;  
    else  
        current_speed = PWM_MAX;  
    int display_speed = current_speed / 10;  
    // Mesaj formatını sadeleştirelim  
    memset(msg_buffer, 0, sizeof(msg_buffer)); // Buffer'ı temizle  
    int len = sprintf(msg_buffer, "Hiz: %d%%\r\n", display_speed);  
  
    HAL_UART_Transmit(&huart2, (uint8_t*)msg_buffer, len, 100);  
    bt_rx = '0';  
}
```

```
void Motor_Speed_Down(void)
```

```
{  
    if (current_speed >= PWM_MIN + PWM_STEP)  
        current_speed -= PWM_STEP;  
    else  
        current_speed = PWM_MIN;  
  
    int display_speed = current_speed / 10;  
    memset(msg_buffer, 0, sizeof(msg_buffer));  
    int len = sprintf(msg_buffer, "Guncel PWM: %d%%\r\n", display_speed);  
    HAL_UART_Transmit(&huart2, (uint8_t*)msg_buffer, len, 100);  
    bt_rx = '0';  
}
```